

Indexing

What is an Index?

- A simple index is a table containing an ordered list of keys and reference fields.
 - e.g. the index of a book
- Simple indexes are represented using simple arrays of structures that contain keys and references.
- In general, indexing is another way to handle the searching problem.

Uses of an index

- An index lets us **impose order on a file** without rearranging the file.
- Indexes provide **multiple access paths** to a file.
 - e.g. library catalog providing search for author, book and title
- An index can provide **keyed access to variable-length record** files.

A simple index for a pile file

	Label	ID	Title	Artist
17	LON	2312	Symphony No.9	Beethoven Giulini
62	RCA	2626	Romeo and Juliet	Prokofiev Maazel
117	WAR	23699	Nebraska ...	
152	ANG	3795	Violin Concerto ...	

Address of
record

Primary key = company label + record ID.

- Index is sorted (in main memory).
- Records appear in file in the order they entered.

Index array:

Key	Reference
ANG3795	152
LON2312	17
RCA2626	62
WAR23699	117

- How to search for a recording with given LABEL ID?
 - Binary search in the index and then seek for the record in position given by the reference field.

Operations to maintain an indexed file

- Create the original empty index and data files.
- Load the index file into memory before using it.
- Rewrite the index file from memory after using it.
- Add data records to the data file.
- Delete records from the data file
- Update records in the data file.
- Update the index to reflect changes in the data file

Rewrite the index file from memory

- When the data file is closed, the index in memory needs to be written to the index file.
- An important issue to consider is what happens if the rewriting does not take place (e.g. power failures, turning machine off, etc.)
- Two important safeguards:
 - Keep a status flag in the header of the index file.
 - If the program detects the index is out of date it calls a procedure that reconstructs the index from the data file.

Record Addition

1. Append the new record to the end of the data file.
2. Insert a new entry to the index in the right position.
 - needs rearrangement of the index: we have to shift all the entries with keys that are larger than the inserted key and then place the new entry in the opened space.
- Note: this rearrangement is done in the main memory.

Record Deletion

- This should use the techniques for reclaiming space in files when deleting records from the data file. We must delete the corresponding entry from the index:
 - Shift all records with keys larger than the key of the deleted record to the previous position;
or
 - Mark index entry as deleted.

Record Updating

There are two cases to consider:

1. The update changes the value of the key field:
 - Treat this as a deletion followed by an insertion
2. The update does not affect the key field:
 - If record size is unchanged, just modify the data record. If record size is changed treat this as a delete/insert sequence.

Indexes too large to fit into Memory

- If the index does not fit in memory, we have the following problems:
 - Binary searching of the index is done on disk, involving several “seeks”.
 - Index rearrangement requires shifting on disk.
- Two main alternatives:
 1. Hashed organization (when access speed is a top priority)
 2. Tree-structured (multi-level) index such as B+trees.

Indexing by Multiple Keys

- We could build additional indexes for a file to provide multiple views of a data file.
 - e.g. Find all recordings of Beethoven's work.
- LABEL ID is a **primary key**.
- There may be **secondary keys**: title, composer, artist.
- We can build **secondary key indexes**.

Composer index:

Secondary key	Primary key
Beethoven	ANG3795
Beethoven	DG139201
Beethoven	DG18807
Beethoven	RCA2626
Corea	WAR23699
Dvorak	COL31809
Prokofiev	LON2312

- Note that secondary key reference is to the primary key rather than to the byte offset.

Record Addition

- When adding a record, an entry must also be added to the secondary key index.
- Insertion is similar to adding entries to the primary key index, i.e. shifting may be necessary.
- There may be duplicates in secondary keys.
 - Keep duplicates in sorted order of primary key.

Record Deletion

- This implies removing all references to the record in the primary index and in all secondary indexes. But this is too much rearrangement.
- Alternative:
 - Delete the record from the data file and the primary index file reference to it. Do not modify the secondary index files.
 - When accessing the file through a secondary key, the primary index file will be checked and a deleted record can be identified.

Record Updating

There are three types of updates:

1. Update changes to the secondary key:
 - We have to rearrange the secondary key index to stay in order.
2. Update changes the primary key:
 - Update and reorder the primary key index; update the references to primary key index in the secondary key indexes (it may involve some re-ordering of secondary indexes if secondary key occurs repeated in the file)
3. Update confined to other fields:
 - This won't affect secondary key indexes. The primary key index may be affected.

Retrieval using combinations of secondary keys

- Secondary key indexes are useful in allowing the following kinds of queries:
 - Find all recordings of Beethoven's work.
 - Find all recordings titled "Violin concerto"
 - Find all recordings with composer Beethoven and title Symphony No.9.
- Boolean operators "and", "or" can be used to combine secondary key values to qualify a request.

Example

- The last query is executed as follows:

Matches from composer index	Matches from title index	Matched list (logical “and”)
ANG3795	ANG3795	ANG3795
DG139201	COL31809	DG18807
DG18807	DG18807	
RCA2626		

Improving 2ndary index Structure: Inverted Lists

- Two difficulties found in the proposed secondary index structures:
 - We have to rearrange the secondary index file every time a new record is added to the file, even if the new record is for an existing secondary key.
 - If there are duplicate secondary keys then the secondary key field is repeated for each entry, wasting space.
- Solution : Inverted Lists

Secondary key Index file

0	Beethoven	3
1	Corea	2
2	Dvorak	5
3	Prokofiev	7

LABEL ID List File

0	LON2312	-1
1	RCA2626	-1
2	WAR23699	-1
3	ANG3795	6
4	DG18807	1
5	COL31809	-1
6	DG139201	4
7	ANG36193	0